



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»
РТУ МИРЭА

ДИСЦИПЛИНА	Программирование управляемых технических систем (укажите полное наименование дисциплины без сокращений)
ИНСТИТУТ	ПТИП (укажите название учебного института)
КАФЕДРА	Индустриального программирования (укажите полное наименование кафедры)
ВИД УЧЕБНОГО МАТЕРИАЛА	Материалы для практических занятий (в соответствии с пп.1-11)
ПРЕПОДАВАТЕЛЬ	Закожурников Сергей Сергеевич (фио)
СЕМЕСТР	2, 2026 (указать семестр обучения, учебный год)

Практическая работа

Изучение триггеров

Триггеры правил

Прежде чем правило начнёт выполняться его необходимо активировать. Есть следующие категории триггеров (активаторов) правил:

1. Триггеры по элементам (Items): реагируют на события на шине событий openHAB, т.е. команды и обновления статуса для элементов;
2. Триггеры по группе: реагируют на события на шине событий openHAB относящиеся к элементам входящим в указанную группу;
3. Триггеры по времени: срабатывают в определенное время, например, в полночь, каждый час и т. д.;
4. Системные триггеры: реагируют на определенные системные состояния;
5. Триггеры по вещам (Things): реагируют на статус вещи, т.е. переходы с ONLINE на OFFLINE и наоборот.

Триггеры по элементам

Можно прослушивать команды для конкретного элемента, при обновлениях статуса или об изменении статуса (обновление может оставить статус без изменений). Можно реагировать только на конкретную команду/статус или любое изменение. Вот синтаксис для всех этих случаев описан в таблице 4 (части в квадратных скобках являются необязательными).

Таблица 4

<code>Item <item> received command [<command>]</code>
<code>Item <item> received update [<state>]</code>
<code>Item <item> changed [from <state>] [to <state>]</code>

При написании правил через пользовательский интерфейс обратите внимание, что не все поля обязательно заполнять. Для того чтобы реагировать не на конкретное состояние, а на любое изменение. А далее значение можно сравнить с числом в логическом блоке условий «But only if» ниже в форме. Это может быть полезно для реакции на превышение параметров с датчиков заранее заданных значений, таких как освещённость, температура или влажность, например. Так же бывает полезно анализировать появление конкретного состояния элемента игнорируя его предыдущее состояние (для этого можно использовать конструкцию `changed to` или просто не указывать лишнее состояние).

Упрощенно объяснить различия между `command` и `update` можно объяснить так: `command` — управление исполнительными устройствами (освещением, воротами, жалюзи и т.п.), `update` — изменение состояния свойственное датчикам. При использовании срабатывания по `received command` правило может начать исполняться до того, как элемент поменяет своё состояние. Поэтому, если правилу важно знать, какая конкретно команда была отправлена следует использовать неявную

переменную receivedCommand вместо <ItemName>.state.

Триггеры по группе

Как и в случае с триггерами на основе событий, описанными выше, вы можете прослушивать команды, обновления статуса или изменения статуса для членов данной группы. Вы также можете решить, хотите ли вы реагировать только на определенную команду / статус или любое изменение. Все неявные переменные заполняются с помощью элемента, вызвавшего событие. Неявная переменная triggeringItem заполняется элементом, который вызвал срабатывание правила, это описано в таблице 5.

Таблица 5

Member of <group> received command [<command>]
Member of <group> received update [<state>]
Member of <group> changed [from <state>] [to <state>]

Триггер по группе работает только с элементами, являющимися прямыми участниками группы. Он не реагирует на элементы вложенных групп. Так же как в случае с триггерами по событию при использовании received command правило может начать выполняться до того, как поменяется состояние. Если правилу необходимо знать, какая была отправлена команда, то нужно использовать неявную переменную receivedCommand вместо <ItemName>.state.

Триггеры по времени

Вы можете использовать некоторые predetermined выражения для таймеров или использовать выражение cron:

Time is midnight

Time is noon

Time cron "<выражение cron>"

Выражение cron имеет форму шести или семи полей:

Секунды

Минуты

Часы

День месяца

Месяц

День недели

Год (необязательное поле)

Можно использовать FreeFormatter.com для создания выражений cron.

Системные триггеры

Существует один системный триггер, описанный ниже в таблице 6.

Таблица 6

Триггер	Описание
System started	System started запускается при запуске openHAB. В openHAB версии 2 System started также срабатывает после изменения файла правил, содержащего триггер System started, или после изменения элементов в файле .items.

Вы можете использовать системный триггер ‘System started’ для установки начальных значений если они ещё не установлены. Пример описан в таблице 7.

Таблица 7

```
rule "Инициализация Speedtest"
when
    System started
then
    createTimer(now.plusSeconds(30), [|
        if (Speedtest_Summary.state == NULL ||
Speedtest_Summary.state == "")
Speedtest_Summary.postUpdate("unknown")
    |])
end
```

Триггеры по вещам

Ваши правила могут предпринимать действия, основанные на обновлениях статуса или изменениях статуса описанных вещей (Things). Вы можете решить, хотите ли вы отслеживать только определенный статус или любой статус. Синтаксис для всех этих случаев (части в квадратных скобках необязательны) описан в таблице 8.

Таблица 8

```
Thing <thingUID> received update [<status>]
Thing <thingUID> changed [from <status>] [to <status>]
```

Статус, используемый в триггере и сценарии, представляет собой строку (без кавычек). Вы можете найти все возможные значения статуса в [Thing Status](#). Чтобы узнать, как получить статус объекта в сценарии, обратитесь к разделу «[Действие по статусу объекта](#)». Идентификатор thingUID, присвоенный Вещи вручную в вашей конфигурации или автоматически во время автоматического обнаружения. Вы можете найти его в пользовательском интерфейсе или с удаленной консоли Karaf. Например, одно устройство z-wave может быть «zwave: device: c5155aa4: node14». Необходимо помещать thingUID в кавычки если оно содержит специальные символы.

Триггеры по каналам

Некоторые надстройки предоставляют триггерные каналы. По сравнению с другими типами каналов, канал запуска предоставляет информацию о дискретных событиях, но не предоставляет непрерывную

информацию о состоянии.

Ваши правила могут предпринимать действия на основе триггерных событий, генерируемых этими каналами. Вы можете решить, хотите ли вы поймать только определенный или любой триггер, который предоставляет канал. Вот синтаксис для этих случаев выглядит следующим образом: необходимо помещать `triggerChannel` в кавычки, если оно содержит специальные символы.

`Channel "<triggerChannel>" triggered [<triggerEvent>]`

`triggerChannel` — это идентификатор конкретного канала.

Информацию о создаваемых каналах можно найти в документации надстройки (Add-on). Не существует списка возможных значений `triggerEvent`, так как это зависит от конкретной реализации надстройки. Если необходимо знать какое именно событие произошло необходимо использовать неявную переменную `receivedEvent`, как показано в таблице 9.

Таблица 9

```
rule "Начинаем пробуждение на рассвете"
when
    Channel "astro:sun:home:rise#event" triggered START
then
    ...
end
```

Сценарии

Язык выражений, используемый в сценариях, тот же, что используется в языке Xtend. Синтаксис очень похож на Java, но имеет много хороших функций, позволяющих писать лаконичный код. Это особенно эффективно при работе с коллекциями. Что делает его подходящим для openHAB с технической точки зрения, так это то, что нет необходимости компилировать сценарии, поскольку они могут быть интерпретированы во время выполнения.

Чтобы иметь возможность делать что-то полезное со сценариями, openHAB предоставляет доступ:

- о всем определенным элементам, чтобы вы могли легко получить к ним доступ по их имени
- ко всем перечисленным состояниям / командам, например, ON, OFF, DOWN, INCREASE и т. д.
- ко всем стандартным действиям

Комбинируя эти функции, вы можете легко написать такой код, как в таблице 10.

Таблица 10

```
if (Temperature.state < 20) {
    Heating.sendCommand(ON)
}
```

Задание:

1. Создать триггер правил
2. Создать триггер по элементам
3. Создать триггер по группе
4. Создать триггер повремени
5. Создать системный триггер
6. Создать триггер по вещам
7. Создать триггер по каналам.